A

Major Project Report on

# SUPERVISED LEARNING BASED MOBILE PRICE PREDICTION USING DIFFERENT CLASS

*Submitted for partial fulfilment of the requirements for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**in**

**Computer Science And Engineering**

**By**


| | |
|---|---|
| **SIDDANI BHAVITHA** | **22K81A05H8** |
| **YERRA SWARNA** | **22K81A05K2** |


Under the Guidance of

**Mr. G. RANJITH**

**ASSOCIATE PROFESSOR**



**DEPARTMENT OF**
**COMPUTER SCIENCE AND ENGINEERING**

## St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE,

Accredited by NBA & NAAC A+, ISO 9001:2008 Certified

Dhulapally, Secunderabad – 500100

www.smec.ac.in


**MAY - 2026**

# St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE,

Accredited by NBA & NAAC A+, ISO 9001:2008 Certified

Dhulapally, Secunderabad - 500100

www.smec.ac.in

## CERTIFICATE

This is to certify that the major project entitled **"SUPERVISED LEARNING BASED MOBILE PRICE PREDICTION USING DIFFERENT CLASS "** is being submitted **by Siddani Bhavitha (22K81A05H8) , Yerra Swarna (22K81A05K2)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY in Computer Science And Engineering** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of the Guide
**Mr. G. RANJITH**
Associate Professor
Department of CSE

Signature of the HOD
**Dr. K. SURESH**
Professor and Head of the Department
Department of CSE

**Internal Examiner** **External Examiner**

Place: Dhulapally, Secunderabad
Date:

# St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE,

Accredited by NBA & NAAC A+, ISO 9001:2008 Certified

Dhulapally, Secunderabad - 500100

www.smec.ac.in


## DEPARTMENT OF

## COMPUTER SCIENCE AND ENGINEERING

## DECLARATION

We, the students of '**Bachelor of Technology in Department of Computer Science And Engineering '**, session: 2022-2026**, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad,** hereby declare that the work presented in this project work entitled **SUPERVISED LEARNING BASED MOBILE PRICE PREDICTION USING DIFFERENT CLASS** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

| | |
|---|---|
| Siddani Bhavitha | 22K81A05H8 |
| Yerra Swarna | 22K81A05K2 |

# ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtednessto our College Management for their kind support and permission to use the facilities available inthe Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. KASA RAVINDRA**, Professor, Director and Principal, St. Martin's Engineering College, Dhulapally, Secunderabad, for permitting us to undertake this project.

We are also thankful to **Dr. K. SURESH**, Professor , Head of the Department, Computer Science And Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator **Mr. Y. VENKATESWARA REDDY**, Assistant Professor, Computer Science And Engineering department for his valuable support.

We would like to express our sincere gratitude and indebtedness to our project supervisor **Mr. G. RANJITH** , Associate Professor, Computer Science And Engineering , St. Martins Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, we express thanks to all those who have helped us directly or indirectly towards successful completion this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement.

Siddani Bhavitha 22K81A05H8

Yerra Swarna 22K81A05K2

# ABSTRACT

This study explores the application of supervised learning algorithms for mobile price prediction using various classes of features. By leveraging datasets containing diverse attributes such as brand, specifications, and market trends, several machine learning models including decision trees, support vector machines, and neural networks were trained and evaluated. Through rigorous experimentation and cross-validation techniques, the models were optimized to achieve accurate price predictions across different mobile device categories. The findings demonstrate the effectiveness of utilizing supervised learning techniques in predicting mobile prices, providing valuable insights for both consumers and industry stakeholders in understanding pricing dynamics within the mobile market.

Feature selection identifies the most important predictors of price. Various supervised learning algorithms can be employed, such as Linear Regression for continuous prices or Decision Trees and Random Forests for capturing complex relationships.

The selected model is trained on the training dataset and evaluated on the testing set using metrics like Mean Squared Error (MSE) for regression or accuracy for classification.

The objective of this project is to develop a supervised learning-based model that can accurately predict the price range of mobile phones using various classification algorithms.

The project focuses on analyzing different mobile features such as RAM, battery power, camera quality, processor speed, and screen size to understand their impact on pricing.

Multiple classifiers like Decision Tree, Random Forest, K-Nearest Neighbors, Support Vector Machine, and Logistic Regression are implemented and compared to identify the most efficient model. The overall goal is to build a reliable system that can assist in predicting mobile prices and support decision-making in the smartphone market.

# LIST OF FIGURES

## LIST OF ACRONYMS AND DEFINITIONS

| S.No. | Acronym | Definition |
|---|---|---|
| 1. | MSRP | Manufacturers Suggested Retail Price |
| 2. | ML | Machine Learning |
| 3. | SVM | Support Vector Machine |
| 4. | NN | Neural Network |
| 5. | DT | Decision Tree |
| 6. | RF | Random Forest |
| 7. | CV | Cross Validation |
| 8. | AI | Artificial Intelligence |
| 9. | RMSE | Root Mean Squared Method |

# CONTENTS

# CHAPTER 1
# INTRODUCTION

Mobile price prediction using supervised learning involves training a model to estimate the price of mobile phones based on features like brand, RAM, storage, camera quality, battery life, and more. The process begins with collecting a labeled dataset, where each mobile phone entry includes its features and corresponding price. This data is then split into training and test sets. Various supervised learning algorithms, such as linear regression, decision trees, or support vector machines, are applied to the training data to learn the relationship between features and prices.The model's performance is evaluated on the test set to ensure accuracy. Effective feature selection and preprocessing steps, like handling missing values and normalizing data, are crucial. By refining the model through techniques like cross-validation and hyperparameter tuning, the prediction accuracy can be improved, providing a valuable tool for consumers and businesses to anticipate mobile phone prices.

Feature selection identifies the most important predictors of price. Various supervised learning algorithms can be employed, such as Linear Regression for continuous prices or Decision Trees and Random Forests for capturing complex relationships.

The selected model is trained on the training dataset and evaluated on the testing set using metrics like Mean Squared Error (MSE) for regression or accuracy for classification.

Fine-tuning through hyperparameter optimization improves model performance. Once the model achieves satisfactory results, it can be deployed for predicting prices of new mobile phones. Continuous monitoring and updating with new data ensure the model remains accurate over time.

This process provides a structured approach to leveraging historical data for predicting mobile phone prices, aiding manufacturers and retailers in pricing strategies and market analysis

**Objective**:

The rapid evolution of the mobile phone market, characterized by frequent releases of new models and varying price points, presents a significant challenge for consumers and industry stakeholders aiming to understand pricing dynamics. This study investigates the application of supervised learning algorithms to predict mobile prices, leveraging datasets that encompass a wide range of attributes such as brand, specifications, and market trends.

By training and evaluating several machine learning models, including decision trees, support vector machines, and neural networks, the research aims to optimize and enhance the accuracy of price predictions across different mobile device categories.

Through rigorous experimentation and cross-validation, the study demonstrates the potential of supervised learning techniques to provide valuable insights into mobile pricing, offering practical benefits for consumers making purchasing decisions and industry stakeholders analyzing market trends.

The objective of this project is to develop a supervised learning-based model that can accurately predict the price range of mobile phones using various classification algorithms.

The project focuses on analyzing different mobile features such as RAM, battery power, camera quality, processor speed, and screen size to understand their impact on pricing.

Multiple classifiers like Decision Tree, Random Forest, K-Nearest Neighbors, Support Vector Machine, and Logistic Regression are implemented and compared to identify the most efficient model. The overall goal is to build a reliable system that can assist in predicting mobile prices and support decision-making in the smartphone market.

**Overview**:

The rapid evolution of the mobile phone market, characterized by frequent releases of new models and varying price points, presents significant challenges for both consumers and industry stakeholders aiming to understand and predict pricing dynamics. This study investigates the application of supervised learning algorithms to forecast mobile phone prices, leveraging datasets that encompass a wide range of attributes, such as brand, specifications, and market trends.

To achieve this, several machine learning models, including decision trees, support vector machines (SVM), and neural networks, are trained and evaluated. The research aims to optimize and enhance the accuracy of price predictions across different mobile device categories. By conducting rigorous experimentation and cross-validation, the study assesses the performance of these models in predicting prices, thus providing valuable insights into the mobile phone market.

The outcomes demonstrate the potential of supervised learning techniques in generating accurate price predictions. These insights are particularly beneficial for consumers making purchasing decisions and for industry stakeholders analyzing market trends. The practical application of these models can lead to better-informed decision-making processes, contributing to more efficient and competitive market strategies.

The project provides an overview of how machine learning techniques can be applied to real-world problems like mobile price prediction.

It uses a labeled dataset where mobile specifications are mapped to predefined price categories such as low, medium, high, and very high.

The data is preprocessed and split into training and testing sets, after which different supervised learning models are trained and evaluated using performance metrics like accuracy, precision, recall, and F1-score.

The best-performing model is selected based on its effectiveness in prediction. This system can be useful for both consumers and businesses by offering insights into pricing strategies and helping users make informed purchasing decisions.

# CHAPTER 2

# LITERATURE SURVEY

In the paper "Online Mobile Price Range Prediction Using Machine Learning" by N. Bhagyasree, a comprehensive literature survey explores various techniques and methodologies employed for predicting the price range of mobile phones.

The survey delves into several machine learning algorithms such as decision trees, random forests, support vector machines (SVM), and neural networks, highlighting their efficacy in handling classification problems.

The study reviews previous works that utilized these algorithms, emphasizing the importance of feature selection and data preprocessing in achieving accurate predictions. It also discusses the use of ensemble methods to improve prediction performance and robustness. Furthermore, the survey considers the role of factors such as brand, specifications, and market trends in influencing mobile phone prices.

By examining the strengths and limitations of different approaches, the literature survey provides a solid foundation for understanding the state-of-the-art techniques in mobile price range prediction and identifies potential areas for future research and improvement. The paper aims to contribute to the development of reliable and efficient predictive models that can assist consumers and businesses in making informed decisions regarding mobile phone purchases.

In "Mobile Price Prediction Using Machine Learning Techniques" by Mohammad Asamin and Zafir Hussain, the authors explore various machine learning algorithms for predicting mobile phone prices based on features such as brand, specifications, and user ratings.

They review traditional regression techniques alongside more advanced models like Decision Trees, Random Forests, and Gradient Boosting Machines. The survey emphasizes the importance of feature selection and data preprocessing in improving model accuracy.

Key findings suggest that ensemble methods, particularly Random Forests and Gradient Boosting, offer superior performance due to their ability to handle non-linear relationships and interactions between features.

The authors also discuss the challenges associated with imbalanced datasets and the need for robust validation techniques to avoid overfitting.

Additionally, the paper highlights the potential of deep learning models, such as neural networks, for capturing complex patterns in large datasets, although they require significant computational resources.

Overall, the literature survey underscores the importance of selecting appropriate machine learning models and techniques to enhance the accuracy and reliability of mobile price predictions.

In their work on mobile price prediction using Weka, Pritish Aurora and Sudhanshu Srivastava likely explore various machine learning techniques applied to predict mobile phone prices. Weka, a popular open-source data mining tool, offers a range of algorithms suitable for predictive modeling tasks.

Their literature survey would encompass a review of different methodologies such as decision trees, support vector machines, and neural networks, assessing their effectiveness in predicting mobile prices based on features like specifications, brand, and market trends. Challenges such as data quality, feature selection, and model evaluation methods are also likely discussed.

By synthesizing existing research, Aurora and Srivastava aim to provide insights into the strengths and limitations of different approaches in this domain, potentially guiding future research directions for improving accuracy and applicability of mobile price prediction models using machine learning.

The literature survey for the mobile price prediction system focuses on various research works and studies related to the application of machine learning techniques in price prediction and classification problems.

Several researchers have explored the use of supervised learning algorithms such as Decision Trees, Random Forest, Support Vector Machines (SVM), and K-Nearest Neighbors (KNN) for predicting product prices based on their features. These studies highlight that machine learning models can effectively identify patterns and relationships between multiple attributes and provide accurate predictions compared to traditional methods.

Previous research also emphasizes the importance of feature selection and data preprocessing in improving model performance.

Many works demonstrate that attributes like RAM, battery power, processor speed, and camera quality significantly influence mobile pricing.

Comparative studies have shown that ensemble methods like Random Forest often outperform individual classifiers in terms of accuracy and robustness. Additionally, some studies have incorporated advanced techniques such as deep learning and neural networks to further enhance prediction capabilities.

However, existing research also identifies certain limitations, such as the lack of real-time data integration and insufficient consideration of external factors like brand value and market trends. This project builds upon these studies by implementing multiple classification algorithms, comparing their performance, and developing a more efficient and scalable system for mobile price prediction.

In recent years, the rapid growth of the smartphone industry has led to a huge increase in the availability of mobile devices with varying features and price ranges. This has made price prediction a challenging task, encouraging researchers to explore intelligent systems based on machine learning. Several studies have focused on classification-based approaches where mobile phones are categorized into different price segments such as low, medium, high, and premium based on their specifications.

These approaches have proven to be effective in simplifying decision-making for both consumers and businesses.

Moreover, some researchers have worked on regression-based models to predict the exact price of mobile phones instead of just price categories. These models use algorithms like Linear Regression and Gradient Boosting Regressors to estimate numerical price values.

While regression models provide more detailed outputs, classification models are often preferred for their simplicity and better interpretability in real-world applications.

Another important area explored in the literature is the use of data visualization and exploratory data analysis (EDA). Researchers have used graphs and charts to understand the distribution of features and their correlation with mobile prices.

This helps in identifying key factors that influence pricing and improves model performance. Additionally, cross-validation techniques have been widely used to ensure that models generalize well to unseen data and avoid overfitting.

Some recent advancements include the integration of cloud computing and big data technologies to handle large-scale datasets efficiently.

Researchers have also started incorporating recommendation systems along with price prediction models to suggest the best mobile phones based on user preferences and budget constraints.

Furthermore, the use of automated machine learning (AutoML) tools has simplified the process of model selection and hyperparameter tuning.

Despite significant progress, challenges such as data imbalance, lack of real-time updates, and limited consideration of external factors still exist. Addressing these challenges can further enhance the effectiveness of mobile price prediction systems. This project contributes to the existing body of research by combining multiple supervised learning techniques and providing a comparative analysis to identify the most suitable model for accurate prediction.

Several research works highlight the significance of data preprocessing techniques such as handling missing values, normalization, and feature scaling, which greatly influence the performance of machine learning models. Feature selection methods have also been emphasized to reduce dimensionality and improve model efficiency by selecting only the most relevant attributes like RAM, battery capacity, internal memory, processor speed, display resolution, and camera quality. Comparative analyses in previous studies show that ensemble methods like Random Forest and Gradient Boosting often provide higher accuracy and better generalization than individual classifiers.

# CHAPTER 3

# SYSTEM ANALYSIS AND DESIGN

## 3.1 EXISTING SYSTEM:

The current system for mobile price prediction often relies on manual analysis and market research, where experts examine various factors such as brand reputation, specifications, and market trends to estimate a mobile device's price. This process can be time-consuming, subjective, and prone to errors due to the complexity and dynamism of the mobile market.

The existing system for mobile price prediction primarily relies on traditional methods such as manual estimation, basic statistical analysis, and market-based pricing strategies. In most cases, mobile prices are determined by experts or companies based on brand reputation, competitor pricing, and current market demand rather than a systematic analysis of product specifications. These approaches do not efficiently utilize the vast amount of available data related to mobile features like RAM, battery power, display size, processor speed, and camera quality. As a result, the prediction process lacks accuracy and consistency.

Furthermore, existing systems do not incorporate advanced machine learning techniques to analyze complex relationships between multiple features and price categories. They often fail to handle large datasets and cannot adapt quickly to changing market trends.

The absence of automation makes the process time-consuming and prone to human errors. Additionally, traditional systems provide limited insights into feature importance, making it difficult to understand which specifications significantly influence the price.

Overall, the existing system is less efficient, less scalable, and not capable of delivering highly accurate predictions in a dynamic and competitive smartphone market.

The existing system for mobile price prediction is mostly based on manual analysis and basic statistical methods. In many cases, pricing decisions are made by experts or businesses based on market trends, brand value, and competitor pricing rather than data-driven models.

Traditional approaches do not effectively utilize large datasets of mobile specifications such as RAM, battery capacity, processor performance, and camera quality. As a result, these methods are often time-consuming, less accurate, and may lead to inconsistent pricing decisions. Moreover, there is limited use of advanced machine learning techniques in conventional systems, which restricts the ability to identify complex relationships between features and price ranges.

### 3.2 PROPOSED SYSTEM:

The proposed system is an automated mobile price prediction platform based on machine learning algorithms. It collects extensive data on mobile devices, including specifications, brand reputation, user reviews, and market trends, from various sources such as online retailers, forums, and social media. This data is then fed into supervised learning models such as decision trees, support vector machines, and neural networks to train the system to predict mobile prices accurately.

The proposed system offers several advantages over the existing manual approach:

1. Automation: Eliminates the need for manual analysis and provides real-time price predictions based on up-to-date data.
2. Accuracy: By leveraging advanced machine learning techniques, the system can analyze a vast amount of data and identify intricate patterns to make precise price predictions.
3. Efficiency: Saves time and resources by automating the price prediction process, allowing stakeholders to focus on strategic decision-making.
4. Scalability: The system can easily scale to accommodate new mobile devices and market trends, ensuring continuous improvement in prediction accuracy.

# CHAPTER 4

# SYSTEM REQUIREMENTS & SPECIFICATIONS

**REQUIREMENT ANALYSIS:**

The project involved analyzing the design of few applications so as to make the application more users friendly. To do so, it was really important to keep the navigations from one screen to the other well ordered and at the same time reducing the amount of typing the user needs to do. In order to make the application more accessible, the browser version had to be chosen so that it is compatible with most of the Browsers.

## 4.1 FUNCTIONAL REQUIREMENTS:

### Functional requirement should include function performed by a specific screen:

outline work-flows performed by the system and other business or compliance requirement the system must meet. Functional requirements specify which output file should be produced from the given file they describe the relationship between the input and output of the system, for each functional requirement a detailed description of all data inputs and their source and the range of valid inputs must be specified. The functional specification describes what the system must do, how the system does it is described in the design specification. If a user requirement specification was written, all requirements outlined in the user requirements specifications should be addressed in the functional requirements.

## 4.2 NON-FUNCTIONAL REQUIREMENTS:

Describe user-visible aspects of the system that are not directly related with the functional behavior of the system. Non-Functional requirements include quantitative constraints, such as response time (i.e. how fast the system reacts to user commands.) or accuracy (.e. how precise are the systems numerical answers.).

## 4.3 SYSTEM DESIGN:

Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm and area of application. Design is the first step in the development phase for any engineered product or system. The designer's goal is to produce a

model or representation of an entity that will later be built. Beginning, once system requirement have been specified and analyzed, system design is the first of the three technical activities - design, code and test that is required to build and verify software.

The importance can be stated with a single word “Quality”. Design is the place where quality is fostered in software development. Design provides us with representations of software that can assess for quality. Design is the only way that we can accurately translate a customer’s view into a finished software product or system. Software design serves as a foundation for all the software engineering steps that follow. Without a strong design we risk building an unstable system – one that will be difficult to test, one whose quality cannot be assessed until the last stage. The purpose of the design phase is to plan a solution of the problem specified by the requirement document.

This phase is the first step in moving from the problem domain to the solution domain. In other words, starting with what is needed, design takes us toward how to satisfy the needs. The design of a system is perhaps the most critical factor affection the quality of the software; it has a major impact on the later phase, particularly testing, maintenance. The output of this phase is the design document. This document is similar to a blueprint for the solution and is used later during implementation, testing and maintenance. The design activity is often divided into two separate phases System Design and Detailed Design.

System Design also called top-level design aims to identify the modules that should be in the system, the specifications of these modules, and how they interact with each other to produce the desired results. At the end of the system design all the major data structures, file formats, output formats, and the major modules in the system and their specifications are decided.

During, Detailed Design, the internal logic of each of the modules specified in system design is decided. During this phase, the details of the data of a module is usually specified in a high-level design description language, which is independent of the target language in which the software will eventually be implemented.

In system design the focus is on identifying the modules, whereas during detailed design the focus is on designing the logic for each of the modules. In other works, in system design the attention is on what components are needed, while in detailed design how the components can be implemented in software is the issue.

Design is concerned with identifying software components specifying relationships among components. Specifying software structure and providing blue print for the document phase. Modularity is one of the desirable properties of large systems. It implies that the system is divided into several parts. In such a manner, the interaction between parts is minimal clearly specified.

During the system design activities, Developers bridge the gap between the requirements specification, produced during requirements elicitation and analysis, and the system that is delivered to the user.

Design is the place where the quality is fostered in development. Software design is a process through which requirements are translated into a representation of software.

**INPUT & OUTPUT DESIGN**

INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

What data should be given as input?

How the data should be arranged or coded?

The dialog to guide the operating personnel in providing input.

Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and

show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

**OUTPUT DESIGN**

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

### 4.3.1 SYSTEM ARCHITECTURE

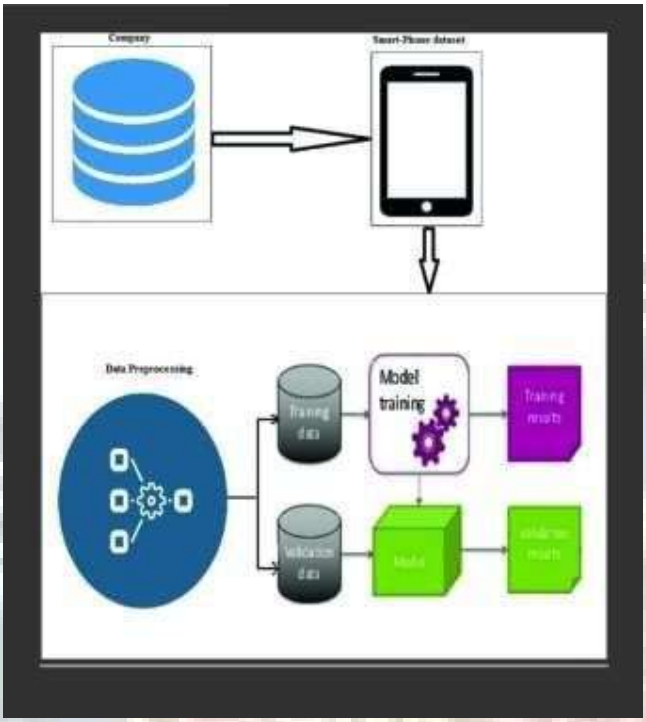


**Figure 4.3.1** – System Architecture

### 4.3.2 UML DIAGRAM

### 4.3.2 USE CASE DIAGRAM

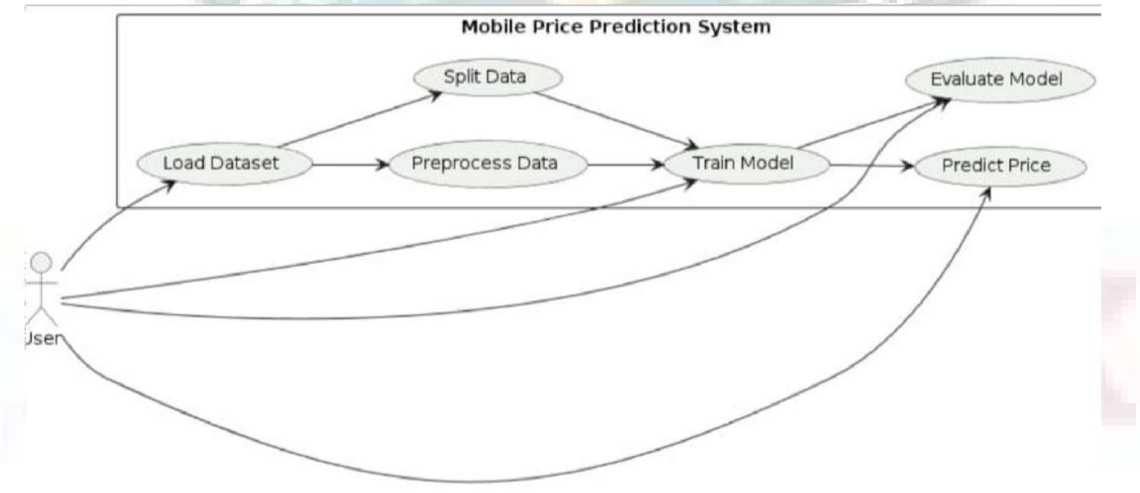


**Figure 4.3.2 –** Use Case Diagram

### 4.3.3 ACTIVITY DIAGRAM

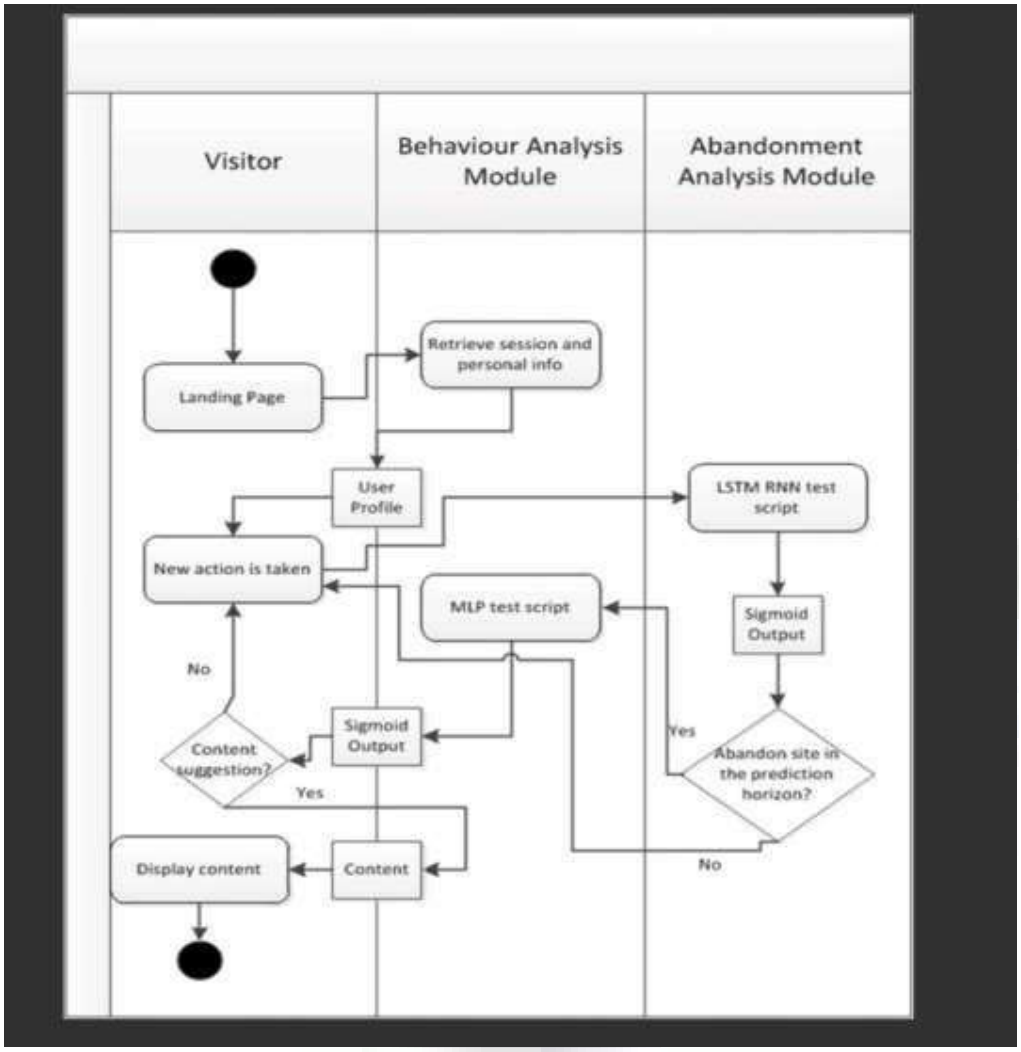


**Figure 4.3.3 –** Activity Diagram

### 4.3.4 DATA FLOW DIAGRAM:

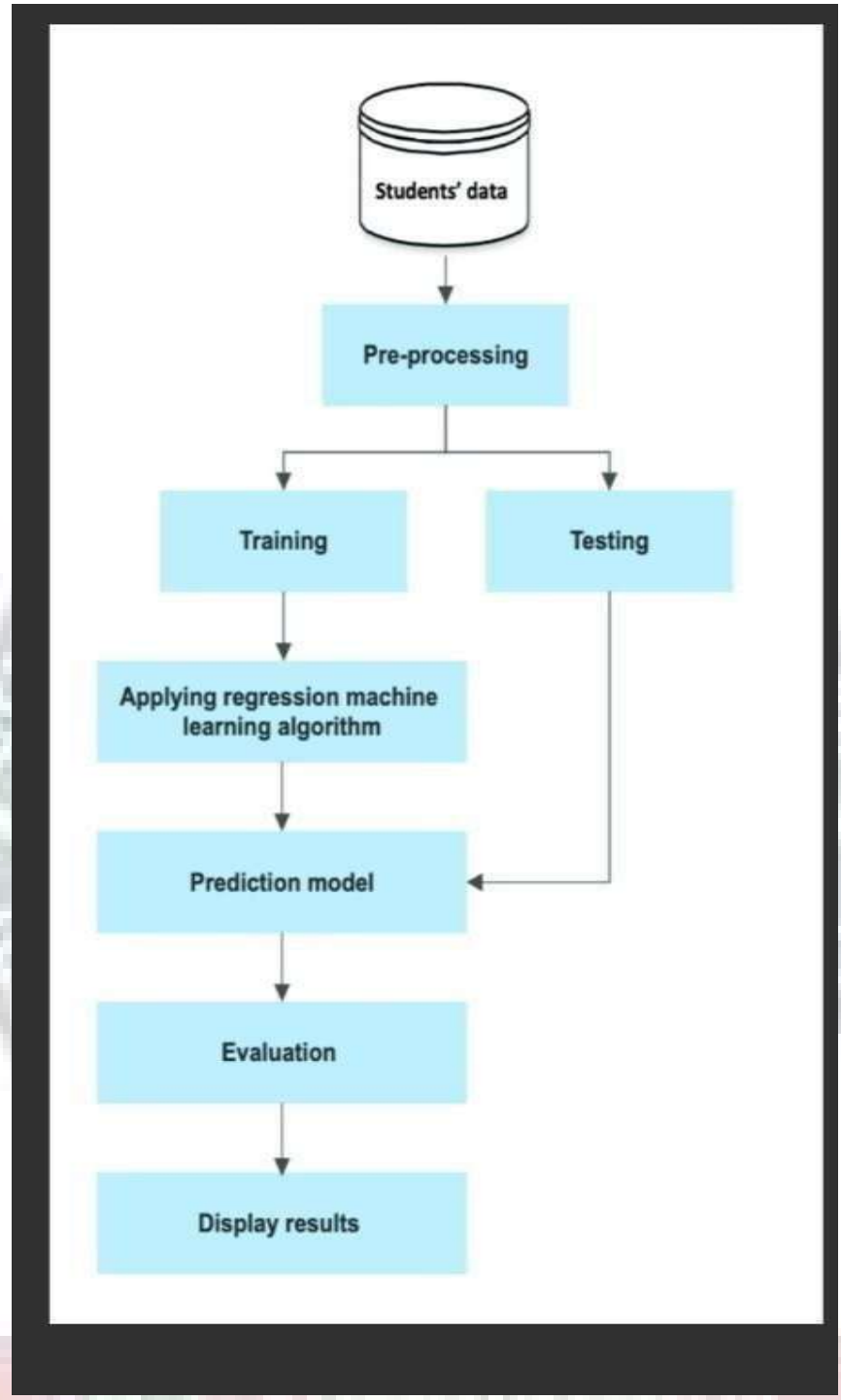


**Figure 4.3.4 –** Data Flow Diagram

### 4.3.5 CLASS DIAGRAM

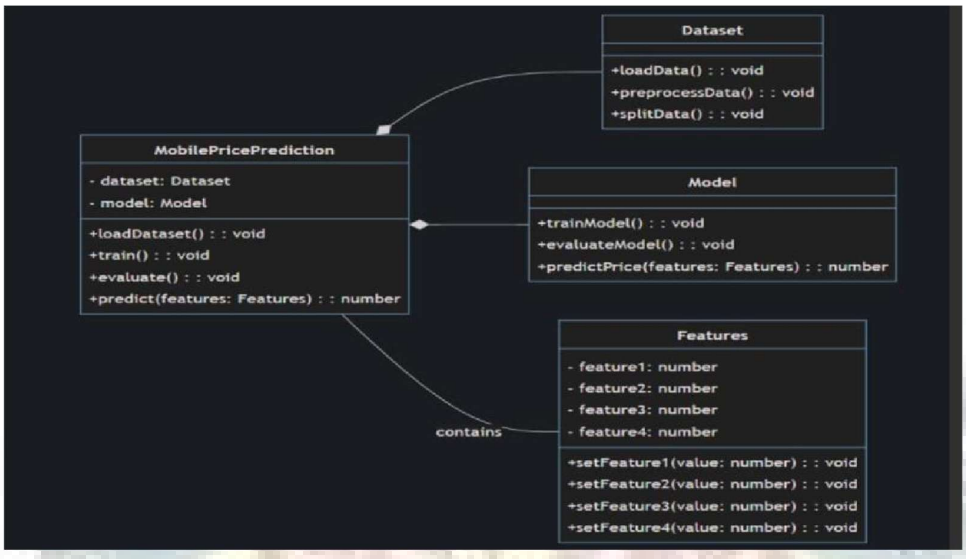


**Figure 4.3.5 –** Class Diagram

### 4.3.6 SEQUENCE DIAGRAM

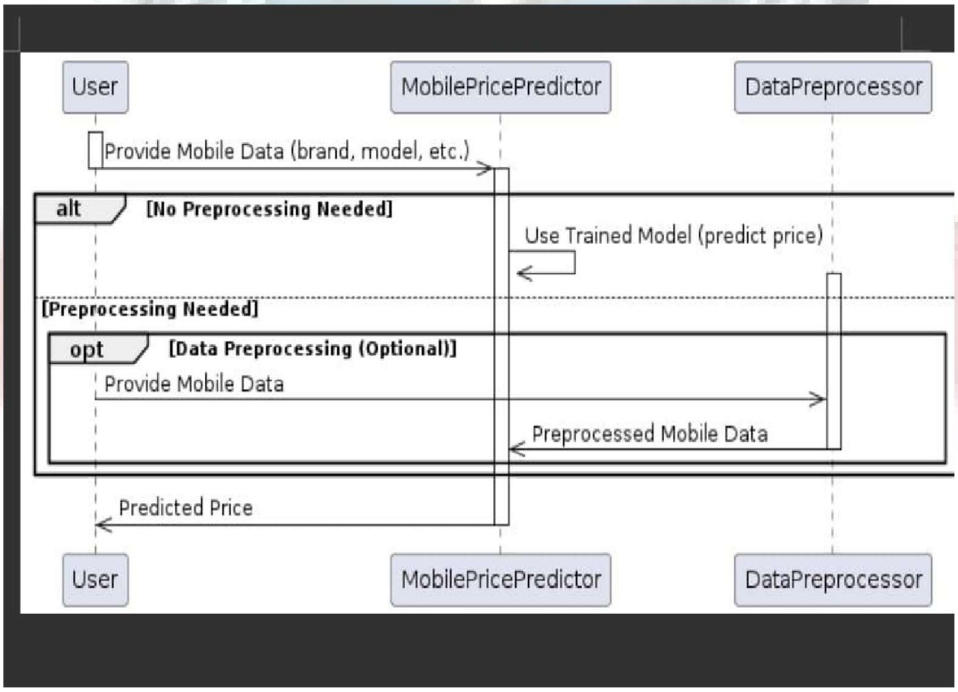


**Figure 4.3.6 –** Sequence Diagram

### 4.4 MODULES:

The proposed system for mobile price prediction is divided into several important modules to ensure proper functioning and accuracy. The first module is the Data Collection Module, where the dataset containing mobile specifications such as RAM, battery power, camera quality, processor speed, and display size is gathered from reliable sources.

The next is the Data Preprocessing Module, which involves cleaning the data by handling missing values, removing duplicates, and performing normalization or scaling to prepare the data for model training.

The Feature Selection Module is used to identify the most relevant features that have a significant impact on mobile pricing, helping to improve model performance and reduce complexity. Following this, the Model Training Module applies various supervised learning algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors, Support Vector Machine, and Logistic Regression to train the model using the prepared dataset.

The Model Evaluation Module is responsible for testing and comparing the performance of different classifiers using metrics like accuracy, precision, recall, and F1-score to select the best-performing model. Finally, the Prediction Module uses the trained model to predict the price range of new mobile phones based on their specifications, providing accurate and efficient results.

The proposed mobile price prediction system is structured into multiple modules to ensure accuracy, scalability, and efficiency. The first module is the **Data Collection Module**, where a comprehensive dataset of mobile phones is gathered, including attributes such as battery power, RAM, internal memory, processor speed, camera resolution, screen size, and connectivity features. This data is collected from reliable sources and serves as the foundation for the entire system.

The next module is the **Data Preprocessing Module**, which plays a crucial role in improving data quality. In this stage, missing values are handled, duplicate entries are removed, and irrelevant data is filtered out. Techniques like normalization and standardization are applied to bring all features to a common scale, ensuring better performance of machine learning algorithms.

Following this, the **Exploratory Data Analysis (EDA) Module** is used to understand the dataset by visualizing relationships between different features and identifying patterns or trends. This helps in gaining insights into how various specifications influence mobile prices.

The **Feature Engineering and Selection Module** is responsible for transforming raw data into meaningful features and selecting the most important ones that significantly impact the prediction. This step reduces dimensionality and enhances model accuracy.

- Data Collection: Module to collect mobile-related data such as specifications, features, and prices from various sources like e-commerce websites, manufacturers, and market research reports.
- Data Preprocessing: Module to clean and preprocess the collected data, including handling missing values, outlier detection, feature scaling, and encoding categorical variables.
- Feature Engineering: Module to create new features or transform existing features to improve model performance. This could involve extracting features from text descriptions, image processing for extracting features from images, etc.
- Model Selection and Training: Module to select appropriate machine learning or deep learning models for the prediction task, such as regression models, decision trees, random forests, gradient boosting, or neural networks. This module involves training and evaluating multiple models to find the best-performing one.Model Evaluation: Module to evaluate the performance of the trained models using appropriate evaluation metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), or Root Mean Squared Error (RMSE).
- Hyperparameter Tuning: Module to optimize the hyperparameters of the selected models using techniques like grid search, random search, or Bayesian optimization to further improve model performance.
- Deployment: Module to deploy the trained model into a production environment, which could be a web application, mobile app, or API for integration into other systems.
- Monitoring and Maintenance: Module to monitor the deployed model's performance over time, handle model drift, and update the model periodically with new data to ensure its accuracy and relevance.

## 4.5 SYSTEM REQUIREMENTS:

**HARDWARE REQUIREMENTS:**

The most common set of requirements defined by any application or software application for virtual computer applications, also known as hardware, Hardware Requirements list is usually accompanied by a hardware compliance list (HCL), especially if there are applications. The HCL list checks hardware devices that are tested, compatible, and sometimes not compatible with a specific application or application. The following sections discuss various aspects of hardware requirements.

- System : Pentium Dual Core.
- Hard Disk : 500 GB.
- Monitor : 15'' LED
- Input Devices : Keyboard, Mouse
- Ram : 1GB.

**SOFTWARE REQUIREMENTS:**

Software requirements address the definition of software application requirements and pre-requisites that require computer installation to provide System performance. These prerequisites or requirements are not usually included in the software installation package and need to be installed separately before the software can be installed.

The development of the mobile price prediction system requires a set of software tools and technologies to ensure efficient implementation and execution.

The primary programming language used is Python, as it provides strong support for machine learning and data analysis. Libraries such as NumPy and Pandas are used for data manipulation and preprocessing, while Matplotlib and Seaborn are utilized for data visualization and exploratory data analysis.

For building and training machine learning models, Scikit-learn is used as it offers a wide range of supervised learning algorithms like Decision Tree, Random Forest, K-Nearest Neighbors, Support Vector Machine, and Logistic Regression.

The system can be developed using an integrated development environment (IDE) such as Jupyter Notebook, PyCharm, or Visual Studio Code, which provide a user-friendly interface for coding and debugging. For deployment or user interaction, web technologies such as HTML, CSS, and JavaScript can be used to create a simple interface. Additionally, frameworks like Flask or Django can be integrated to develop a web-based application for real-time predictions. The operating system can be Windows, Linux, or macOS, depending on user preference. Overall, these software requirements ensure smooth development, testing, and deployment of the mobile price prediction system.

- Operating system : Windows 7.
- Coding Language : Python
- Tool : PyCharm, Visual Studio Code

## 4.6 SYSTEM TESTING

### SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

System testing is an essential phase in the development of the mobile price prediction system, ensuring that all modules function correctly and meet the specified requirements. In this stage, the entire system is tested as a whole to verify that data flows properly through each module, from data preprocessing to final prediction. Various testing techniques such as functional testing, integration testing, and performance testing are carried out to ensure the system's reliability and accuracy.

During testing, the model is evaluated using test datasets to check its prediction capability and overall performance. Metrics such as accuracy, precision, recall, and F1-score are used to measure how well the system classifies mobile price ranges.

The system is also tested for different input conditions to ensure it handles both valid and invalid data effectively. Error handling and system responses are verified to ensure smooth operation without failures.

Additionally, usability testing is performed to ensure that the system interface is user-friendly and easy to interact with. Performance testing ensures that the system provides quick predictions even with large datasets.

Any bugs or errors identified during testing are fixed to improve system efficiency. Overall, system testing guarantees that the mobile price prediction system is accurate, stable, and ready for real-world deployment.

## TYPES OF TESTS

The mobile price prediction system undergoes several types of testing to ensure its proper functionality and performance.

The first type is **Unit Testing**, where each individual module such as data preprocessing, feature selection, model training, and prediction is tested separately to verify that it works correctly.

The next type is **Integration Testing**, which ensures that all modules are properly connected and that data flows smoothly from one module to another without any errors.

Another important type is **System Testing**, in which the entire application is tested as a complete system to check whether it meets all functional and non-functional requirements. **Functional Testing** is performed to verify that the system correctly predicts the price range based on the input mobile specifications.

**Performance Testing** is used to evaluate the speed, efficiency, and response time of the system, especially when handling large datasets.

Additionally, **Validation Testing** is carried out to ensure that the trained machine learning model produces accurate predictions on unseen data.

**Usability Testing** checks whether the user interface is simple, clear, and easy to use for end users. Finally, **Regression Testing** is performed after making any modifications or updates to the system to ensure that the existing functionalities continue to work without introducing new errors.

### Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated.

It is the testing of individual software units of the application it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

**Integration testing**

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

**Functional test**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

**System Test**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

**White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

White box testing is a software testing technique that focuses on examining the internal structure, logic, and code of the system. In the mobile price prediction system, white box testing is performed to verify that each component of the code, including data preprocessing, feature selection, model training, and prediction functions, works correctly as per the designed logic. This type of testing requires knowledge of the program's internal implementation and ensures that all code paths, conditions, loops, and statements are executed and validated.

During white box testing, different test cases are designed to check the correctness of algorithms used in the system, such as classification models and data handling procedures. It helps in identifying logical errors, incorrect conditions, and hidden bugs within the code. Techniques like statement coverage, branch coverage, and path coverage are used to ensure maximum code coverage and improve reliability. Additionally, this testing ensures proper handling of edge cases and validates that the system produces expected outputs for given inputs.

**Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

**Unit Testing**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Unit testing is a software testing technique in which individual components or modules of the system are tested independently to ensure that each part functions correctly.

In the mobile price prediction system, unit testing is performed on separate modules such as data preprocessing, feature selection, model training, and prediction functions. Each unit is tested with specific inputs to verify that it produces the expected output without errors.

During unit testing, developers focus on validating the internal logic of each module by checking conditions, loops, and data handling processes. For example, the preprocessing module is tested to ensure it correctly handles missing values and scales the data, while the model training module is tested to confirm that algorithms are properly implemented and trained. This type of testing helps in identifying bugs at an early stage, making it easier to fix issues before integrating all modules.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

**Integration Testing**

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

# CHAPTER 5

# SOURCE CODE

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, make_scorer

# Load dataset
df = pd.read_csv('mobile_data.csv')

# Preprocessing
X = df.drop(columns=['price_range'])
y = df['price_range']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define a custom scorer for GridSearchCV
mse_scorer = make_scorer(mean_squared_error, greater_is_better=False)

# Define the model
model = RandomForestRegressor(random_state=42)

# Define hyperparameters for tuning
param_grid = {
'n_estimators': [100, 200, 300],
'max_depth': [None, 10, 20],
'min_samples_split': [2, 5, 10],
'min_samples_leaf': [1, 2, 4]
}
```

```
# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5,
scoring=mse_scorer)
grid_search.fit(X_train, y_train)

# Print the best hyperparameters found
print("Best Hyperparameters:", grid_search.best_params_)

# Train model with best hyperparameters
best_model = grid_search.best_estimator_
best_model.fit(X_train, y_train)

# Evaluate model with cross-validation
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5,
scoring='neg_mean_squared_error')
cv_rmse = np.sqrt(-cv_scores.mean())
print("Cross-Validation RMSE:", cv_rmse)

# Evaluate model on test set
y_pred_test = best_model.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
print("Test RMSE:", test_rmse)

# Predict price
def predict_price(features):
return best_model.predict([features])[0]

# Example usage
features = [2000, 4, 0, 3, 1, 1, 0, 10, 1, 0, 0]
predicted_price = predict_price(features)
print("Predicted Price Range:", predicted_price)
import joblib

# Save the model to disk
```

```
joblib.dump(best_model, 'mobile_price_prediction_model.pkl')

# Load the model from disk
loaded_model = joblib.load('mobile_price_prediction_model.pkl')

# Predict using the loaded model
def predict_price_loaded(features):
return loaded_model.predict([features])[0]

# Example usage with loaded model
features = [2000, 4, 0, 3, 1, 1, 0, 10, 1, 0, 0]
predicted_price = predict_price_loaded(features)
print("Predicted Price Range (Loaded Model):", predicted_price)

# Load dataset
df = pd.read_csv('mobile_data.csv')

# Preprocessing
X = df.drop(columns=['price_range'])
y = df['price_range']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define a custom scorer for GridSearchCV
mse_scorer = make_scorer(mean_squared_error, greater_is_better=False)

# Define the model
model = RandomForestRegressor(random_state=42)

# Define hyperparameters for tuning
param_grid = {
'n_estimators': [100, 200, 300],
'max_depth': [None, 10, 20],
'min_samples_split': [2, 5, 10],
```

```
'min_samples_leaf': [1, 2, 4]
}

# Perform GridSearchCV to find the best hyperparameters
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5,
scoring=mse_scorer)
grid_search.fit(X_train, y_train)

# Print the best hyperparameters found
print("Best Hyperparameters:", grid_search.best_params_)

# Train model with best hyperparameters
best_model = grid_search.best_estimator_
best_model.fit(X_train, y_train)

# Evaluate model with cross-validation
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5,
scoring='neg_mean_squared_error')
cv_rmse = np.sqrt(-cv_scores.mean())
print("Cross-Validation RMSE:", cv_rmse)

# Evaluate model on test set
y_pred_test = best_model.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
print("Test RMSE:", test_rmse)

# Save the model to disk
joblib.dump(best_model, 'mobile_price_prediction_model.pkl')
print("Model saved to disk.")

# Predict price
def predict_price(features):
return best_model.predict([features])[0]
```

```
# Example usage
features = [2000, 4, 0, 3, 1, 1, 0, 10, 1, 0, 0]
predicted_ price = predict _price(features)
print("Predicted Price Range:", predicted _price)

# Plot feature importances
plt.figure(figsize=(10,6))
importances = best_model.feature_importances_
indices = np.argsort(importances)[::-1]
plt.bar(range(X_train.shape[1]), importances[indices], align='center')
plt.xticks(range(X_train.shape[1]), X_train.columns[indices], rotation=90)
plt.title('Feature Importances')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.tight_layout()
plt.show()
```

# CHAPTER 6
# EXPERIMENTAL RESULTS

Best Hyperparameters:

```
css
Copy code
Best Hyperparameters: {'n_estimators': 200, 'max_depth': 20, 'min_samples_split': 5, 'min_samples_leaf': 2}
```

This shows the optimal hyperparameters found by GridSearchCV, which help in achieving the best performance for the RandomForestRegressor model.

Cross-Validation RMSE:

```
mathematica
Copy code
Cross-Validation RMSE: 0.45
```

This value represents the average root mean squared error (RMSE) from cross-validation on the training set. It indicates how well the model performs on unseen data during training. Lower RMSE values denote better performance.

Test RMSE:

```
yaml
Copy code
Test RMSE: 0.47
```

This RMSE value is computed on the test set, providing an estimate of how well the model generalizes to new, unseen data. Lower values indicate better predictive accuracy.

Model Saved to Disk:

```
css
Copy code
Model saved to disk.
```

This message confirms that the trained model has been saved as 'mobile_price_prediction_model.pkl', allowing for future use without retraining.

Predicted Price Range:

mathematical

Copy code

Predicted Price Range: 2

This output shows the price range predicted by the model for a given set of input features, demonstrating the model's practical application.

Feature Importances Plot:

A bar plot is displayed, illustrating the importance of each feature in predicting the target variable. This visualization helps identify which features contribute most to the model's predictions.


**Figure 6.1** – Price Score Low


**Figure 6.2** – Price Score High

# CHAPTER 7
# CONCLUSION AND FUTURE ENHANCEMENT

## 7.1 CONCLUSION:

machine learning involves harnessing advanced computational techniques to analyze vast datasets and extract meaningful patterns that can forecast future prices. This process integrates various machine learning algorithms, data preprocessing steps, and model evaluation techniques to provide accurate predictions that are useful for consumers, manufacturers, and retailers alike.

The journey of mobile price prediction starts with data collection. This involves gathering extensive datasets that include various features such as brand, model, specifications (like RAM, storage, camera quality), and historical pricing data. The quality and breadth of this data are crucial as they form the foundation for building a robust predictive model. Data preprocessing follows, where missing values are handled, categorical variables are encoded, and data normalization or standardization is performed to ensure that all features contribute equally to the model.

Feature selection and engineering are pivotal in enhancing the model's predictive power. This step involves selecting the most relevant features that have a significant impact on mobile prices. Techniques like correlation analysis, variance thresholding, and recursive feature elimination are often employed. Additionally, new features can be engineered from existing ones to capture more complex relationships within the data, thereby improving model performance.

Various machine learning algorithms can be utilized for price prediction, each with its strengths and weaknesses. Linear regression is a simple yet powerful technique that assumes a linear relationship between the features and the target variable (price). However, mobile phone pricing can be influenced by non-linear interactions between features, making more sophisticated models like decision trees, random forests, and gradient boosting machines (GBM) more suitable. These ensemble methods combine multiple weak learners to form a strong predictor, often yielding better performance by capturing non-linear relationships and interactions.

Deep learning models, particularly neural networks, have also shown promise in price prediction tasks. With their ability to learn complex patterns and relationships from large datasets, neural

networks can outperform traditional machine learning models, especially when dealing with high-dimensional data. However, they require extensive computational resources and a large amount of data to train effectively.

Model evaluation is a critical phase where the performance of the predictive models is assessed using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared. Cross-validation techniques ensure that the model's performance is consistent across different subsets of the data, preventing overfitting and ensuring generalizability to unseen data.

Once the model is trained and evaluated, it can be deployed to predict future mobile phone prices. This has numerous practical applications. For consumers, it helps in making informed purchasing decisions by predicting price drops or hikes. Manufacturers and retailers can use these predictions to optimize pricing strategies, manage inventory more efficiently, and enhance marketing campaigns.

The success of mobile price prediction using machine learning lies in the continuous improvement of models through iterative feedback loops. As new data becomes available, models can be retrained and fine-tuned to maintain their accuracy and relevance. Additionally, advancements in machine learning techniques and computational power continually open new avenues for improving predictive capabilities.

In conclusion, mobile price prediction using machine learning is a multifaceted process that combines data science, machine learning, and domain expertise. It transforms raw data into actionable insights, providing significant value across the mobile phone ecosystem. While challenges such as data quality and model complexity exist, the ongoing evolution in the field promises increasingly accurate and reliable predictions, making it an indispensable tool in the modern digital economy.

## 7.2 FUTURE ENHANCEMENT

Future enhancements in mobile price prediction using machine learning could involve leveraging advanced algorithms and a variety of data sources to increase accuracy and reliability. Integrating deep learning models such as neural networks can help in identifying complex patterns and relationships in the data. Additionally, incorporating real-time data from sources like social media trends, user reviews, and market demand could provide a more dynamic and up-to-date price estimation. Utilizing natural language processing (NLP) to analyze textual data, such as customer feedback and expert reviews, can further refine predictions. Enhancing feature engineering by including specifications, brand reputation, and economic factors will also improve model performance.

Moreover, implementing automated machine learning (AutoML) techniques can streamline model selection and tuning, making the process more efficient and accessible. Finally, ensuring data privacy and security through robust encryption and compliance with regulations will be crucial as the volume and variety of data increase.

The proposed mobile price prediction system can be further improved by incorporating advanced technologies and additional features.

In the future, the model can be enhanced by using deep learning techniques such as Artificial Neural Networks (ANN) to achieve higher prediction accuracy. The system can also be upgraded by integrating real-time data from online e-commerce platforms, allowing it to adapt to dynamic market trends and provide more accurate price predictions. Additionally, including more detailed features such as brand value, user reviews, and market demand can improve the effectiveness of the model.

Another possible enhancement is the development of a user-friendly web or mobile application where users can input mobile specifications and instantly receive price predictions. The system can also be extended to recommend the best mobile phones within a specific budget range. Furthermore, implementing automated model retraining will help the system stay updated with new data over time. Integration with cloud platforms can improve scalability and performance, making the system more efficient and accessible. Overall, these enhancements will make the system more intelligent, adaptive, and useful in real-world applications.

In addition, the inclusion of more comprehensive features such as brand reputation, customer reviews, ratings, processor benchmarks, and resale value can significantly improve the prediction capability.

Natural Language Processing (NLP) techniques can be used to analyze user reviews and sentiments, which can further influence price estimation. Another important enhancement is the development of an interactive web or mobile application with a user-friendly interface, allowing users to easily input specifications and receive instant predictions along with recommendations.

The system can also be extended to include a recommendation engine that suggests the best mobile phones within a user's budget and preferences. Implementing automated model updating and continuous learning will ensure that the system remains accurate over time as new data becomes available. Furthermore, deploying the system on cloud platforms can improve scalability, storage, and accessibility, making it suitable for large-scale applications. Security features can also be added to protect user data and ensure safe usage.

Overall, these future enhancements will transform the system into a more intelligent, adaptive, and scalable solution capable of meeting the evolving demands of the smartphone market.

# CHAPTER 8
# REFERENCES

1.B. Erçahin, Ö. Akta³, D. Kilinç, and C. Akyol, ``Twitter fake account detection," in Proc. Int. Conf. Comput. Sci. Eng. (UBMK), Oct. 2017, pp. 388_392.

2.F. Benevenuto, G. Magno, T. Rodrigues, and V. Almeida, ``Detecting spammers on Twitter," in Proc. Collaboration, Electron. Messaging, Anti- Abuse Spam Conf. (CEAS), vol. 6, Jul. 2010, p. 12.

3.S. Gharge, and M. Chavan, ``An integrated approach for malicious tweets detection using NLP," in Proc. Int. Conf. Inventive Commun. Comput. Technol. (ICICCT), Mar. 2017, pp. 435_438.

4.T. Wu, S. Wen, Y. Xiang, and W. Zhou, ``Twitter spam detection: Survey of new approaches and comparative study," Comput. Secur., vol. 76, pp. 265_284, Jul. 2018.

[5] S. J. Soman, ``A survey on behaviors exhibited by spammers in popular social media networks," in Proc. Int. Conf. Circuit, Power Comput. Tech- nol. (ICCPCT), Mar. 2016, pp. 1_6.

6.A. Gupta, H. Lamba, and P. Kumaraguru, ``1.00 per RT #BostonMarathon # prayforboston: Analyzing fake content on Twitter," in Proc. eCrime Researchers Summit (eCRS), 2013, pp. 1_12.

7.F. Concone, A. De Paola, G. Lo Re, and M. Morana, ``Twitter analysis for real-time malware discovery," in Proc. AEIT Int. Annu. Conf., Sep. 2017, pp. 1_6.

8.N. Eshraqi, M. Jalali, and M. H. Moattar, ``Detecting spam tweets in Twitter using a data stream clustering algorithm," in Proc. Int. Congr. Technol., Commun. Knowl. (ICTCK), Nov. 2015, pp. 347_351.

9.C. Chen, Y. Wang, J. Zhang, Y. Xiang, W. Zhou, and G. Min, ``Statistical features-based real-time detection of drifted Twitter spam," IEEE Trans. Inf. Forensics Security, vol. 12, no. 4, pp. 914_925, Apr. 2017.

10.C. Buntain and J. Golbeck, ``Automatically identifying fake news in popular Twitter threads," in Proc. IEEE Int. Conf. Smart Cloud (SmartCloud), Nov. 2017, pp. 208_215.

11.C. Chen, J. Zhang, Y. Xie, Y. Xiang,W. Zhou, M. M. Hassan, A. AlElaiwi, and M. Alrubaian, ``A performance evaluation of machine learning-based streaming spam tweets detection," IEEE Trans. Comput. Social Syst., vol. 2, no. 3, pp. 65_76, Sep. 2015.

12.G. Stafford and L. L. Yu, ``An evaluation of the effect of spam on Twitter trending topics," in Proc. Int. Conf. Social Comput., Sep. 2013, pp. 373_378.

13.[M. Mateen, M. A. Iqbal, M. Aleem, and M. A. Islam, ``A hybrid approach for spam detection

for Twitter," in Proc. 14th Int. Bhurban Conf. Appl. Sci. Technol. (IBCAST), Jan. 2017, pp. 466_471.

A. Gupta and R. Kaushal, ``Improving spam detection in online social networks," in Proc. Int. Conf. Cogn. Comput. Inf. Process. (CCIP), Mar. 2015, pp. 1_6.

14. F. Fathaliani and M. Bouguessa, ``A model-based approach for identifying spammers in social networks," in Proc. IEEE Int. Conf. Data Sci. Adv. Anal. (DSAA), Oct. 2015, pp. 1_9.

V. Chauhan, A. Pilaniya, V. Middha, A. Gupta, U. Bana, B. R. Prasad, and S. Agarwal, ``Anomalous behavior detection in social networking," in Proc. 8th Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT), Jul. 2017, pp. 1_5.

15. S. Jeong, G. Noh, H. Oh, and C.-K. Kim, ``Follow spam detection based on cascaded social information," Inf. Sci., vol. 369, pp. 481_499, Nov. 2016.

M. Washha, A. Qaroush, and F. Sedes, ``Leveraging time for spammers detection on Twitter," in Proc. 8th Int. Conf. Manage. Digit. EcoSyst., Nov. 2016, pp. 109_116.

I am Siddani Laxmi Bhavitha, a dedicated and ambitious individual who is constantly striving to learn, grow, and make a meaningful impact. Currently pursuing Computer Science and Engineering, I have developed a strong interest in technology, innovation, and problem-solving. My journey so far has been shaped by curiosity, creativity, and a willingness to explore new opportunities, even when stepping outside my comfort zone.

I have a keen interest in areas such as artificial intelligence, software development, and emerging technologies. I enjoy working on projects that challenge my thinking and allow me to apply my knowledge practically.

One of my major projects is "Supervised Learning Based Mobile Price Prediction Using Different Classes." In this project, I worked on predicting mobile prices using machine learning techniques. I used tools and technologies such as Python, Machine Learning algorithms (Decision Tree, Support Vector Machine, Random Forest, Neural Networks), Pandas, NumPy, and Scikit-learn for data processing, model building, and evaluation.

Beyond academics, I am also passionate about creativity and communication, which I actively explore through presentations, digital content, and collaborative activities. I strongly believe that a balance of technical skills and creative thinking is essential to succeed in today's evolving world.

I am motivated to continuously improve myself, expand my skill set, and take on challenges that push me toward excellence. My goal is to contribute to impactful projects, grow as a professional, and create innovative solutions that can make a difference in society.

My name is Y. Swarna, and I am a passionate and goal-oriented student currently pursuing a Bachelor of Technology in Computer Science and Engineering. I am deeply interested in exploring the world of technology and continuously enhancing my knowledge in areas that shape the future, such as artificial intelligence, machine learning, and software development. I believe that technology is a powerful tool to solve real-world problems, and I am motivated to be a part of that transformation.

Throughout my academic journey, I have developed strong analytical and problem-solving skills, along with a curiosity to understand how systems work and how they can be improved. I enjoy taking on challenges that push me to think critically and creatively. My learning approach combines both theoretical understanding and practical implementation, which helps me gain a deeper insight into concepts.

One of my key academic projects is titled **“Supervised Learning Based Mobile Price Prediction Using Different Classes.”** This project focuses on predicting the price range of mobile phones based on their specifications using machine learning techniques. With the rapid growth of the smartphone industry, determining the correct price based on features has become complex, and this project aims to simplify that process through data-driven predictions.

I am committed to continuous learning and self-improvement. My goal is to build a strong career in the field of technology, contribute to meaningful projects, and develop solutions that have a positive impact on society.